

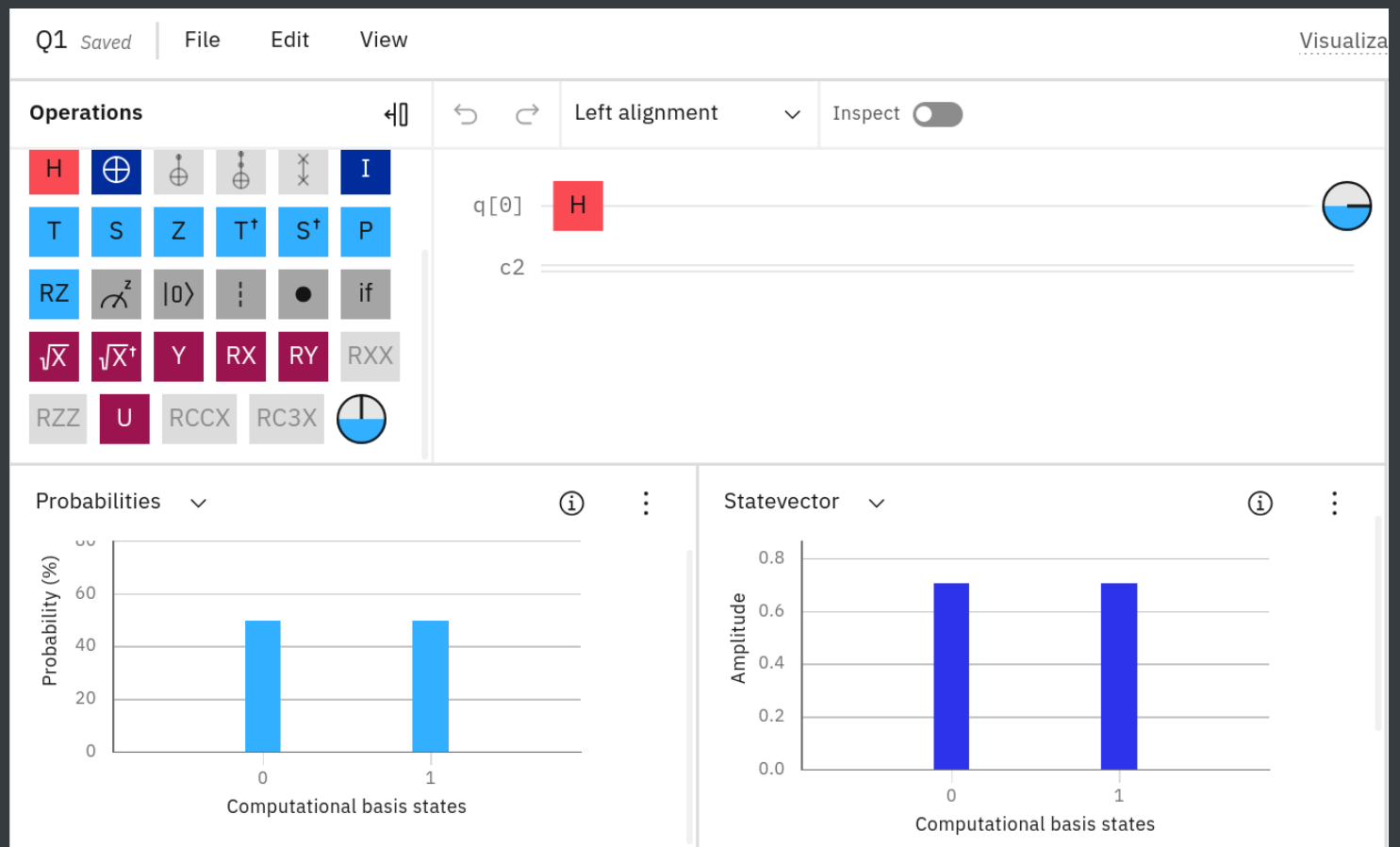
Demo 3

Quantum Computing

Name: Brandon Loh Ming Fong

ID : 47754764

Q1.



```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi

qreg_q = QuantumRegister(1, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)

circuit.h(qreg_q[0])
```

Explanation:

An important step in building qubits is to be able to convert between classical and quantum states

The Hadamard operator H allows us to create the superposition of states of a qubit with its form

$$H = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (6.47)$$

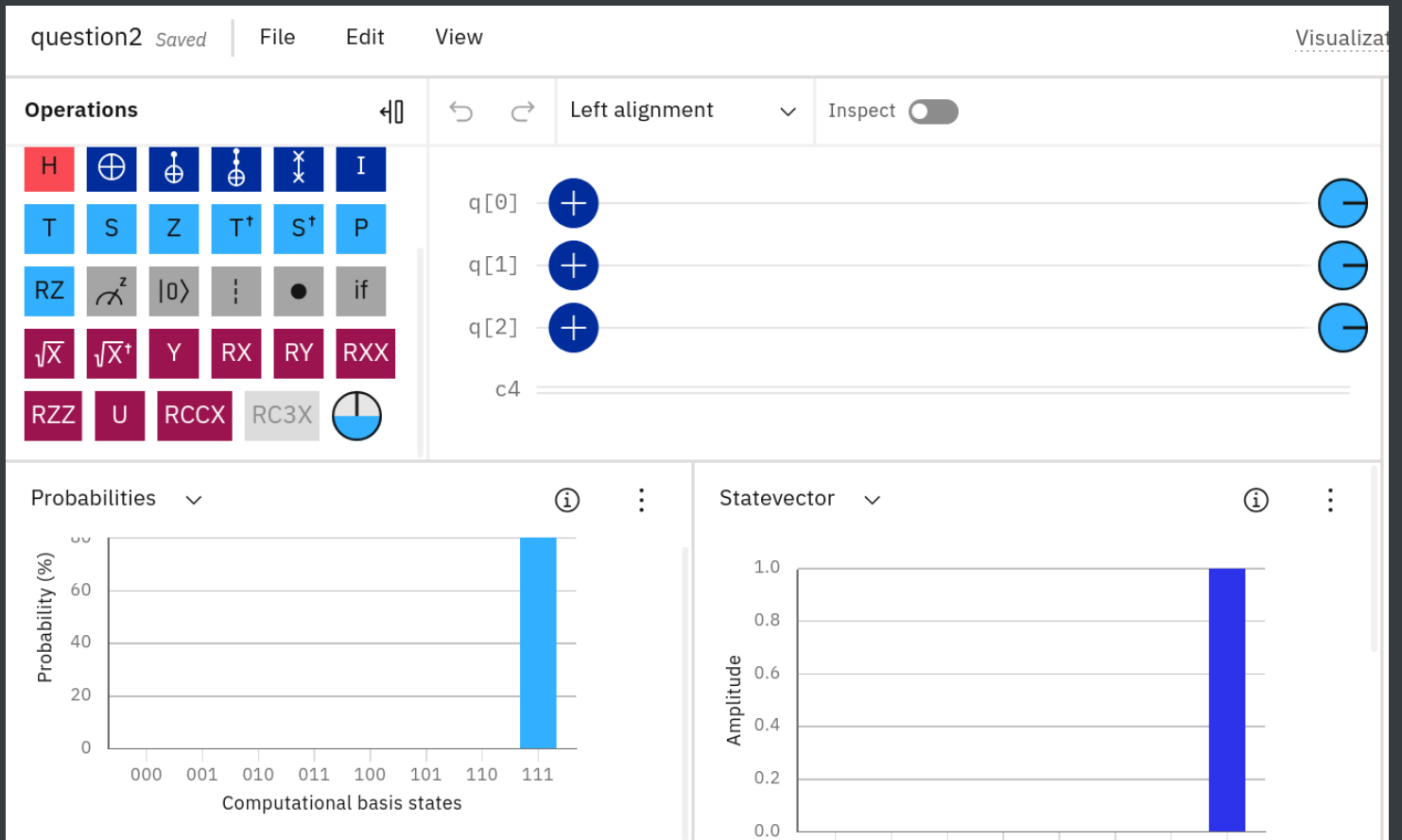
$$\begin{aligned} H|0\rangle &= \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \\ H|1\rangle &= \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} \end{aligned}$$

creates superposition.

This results in a 50% chance of measuring, Dirac notation of '0' or '1', which is key for quantum superposition.

why -1?

Q2.



```

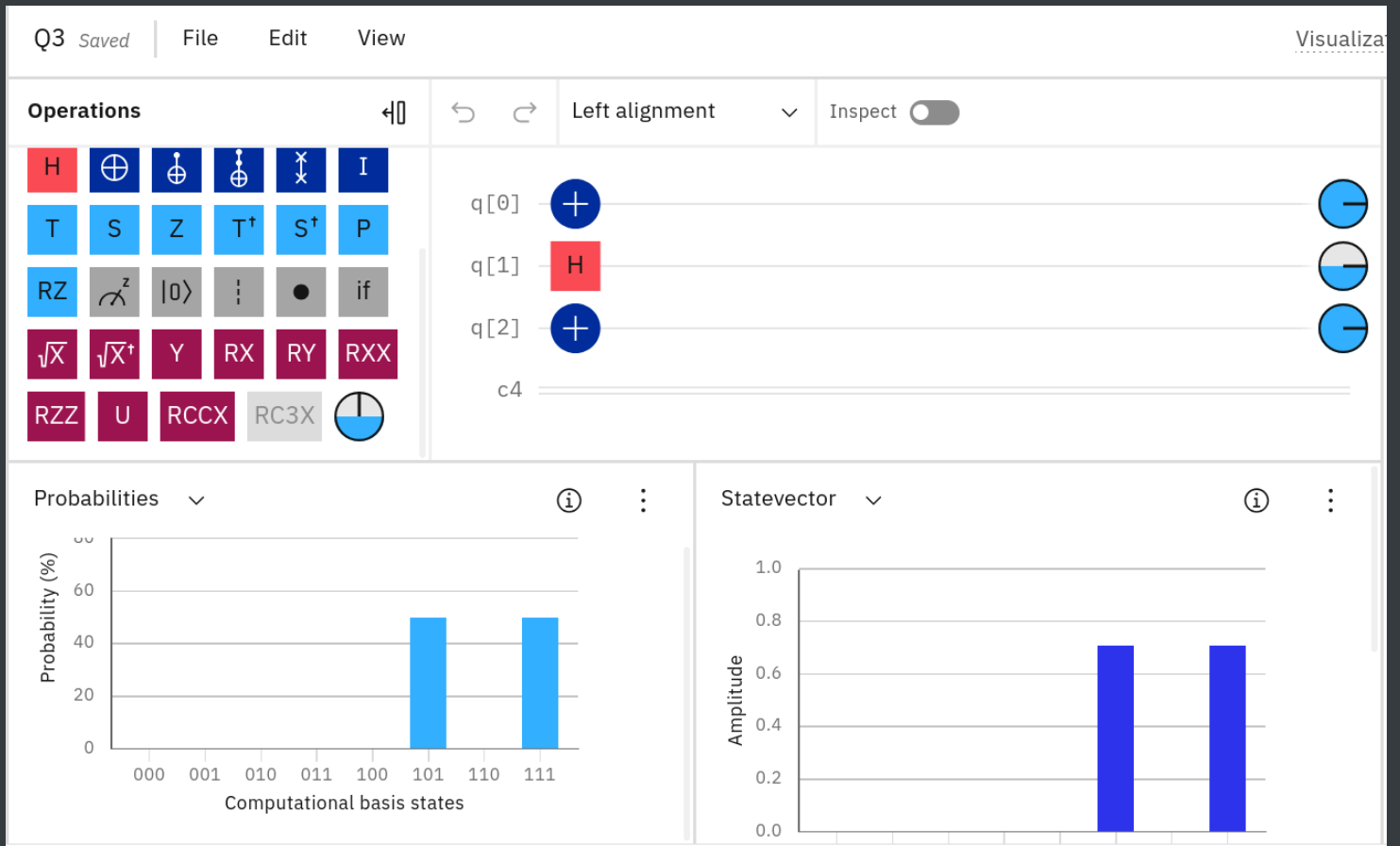
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi

qreg_q = QuantumRegister(3, 'q')
creg_c = ClassicalRegister(4, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)

circuit.x(qreg_q[0])
circuit.x(qreg_q[1])
circuit.x(qreg_q[2])

```

Q3.



```

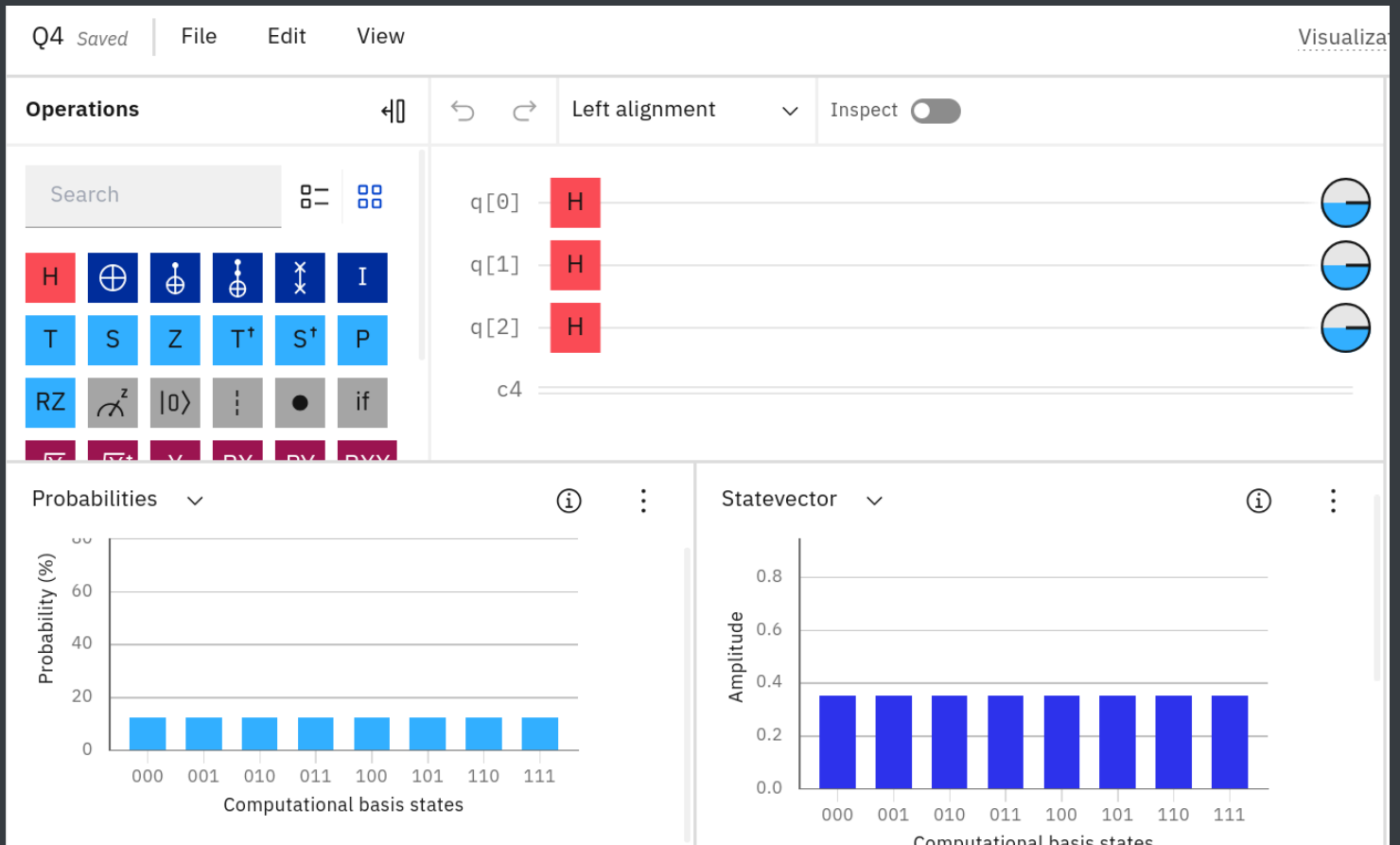
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi

qreg_q = QuantumRegister(3, 'q')
creg_c = ClassicalRegister(4, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)

circuit.x(qreg_q[0])
circuit.x(qreg_q[2])
circuit.h(qreg_q[1])

```

Q4.



```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi
```

```
qreg_q = QuantumRegister(3, 'q')
creg_c = ClassicalRegister(4, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)
```

```
circuit.h(qreg_q[1])
circuit.h(qreg_q[2])
circuit.h(qreg_q[0])
```

Q5.

Reference: Modern Computation by S. S. Chandra pg. 65.

A Bell state is a maximally entangled state of two qubits.

$$\begin{aligned}
 H \otimes I &= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\
 &= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}.
 \end{aligned}$$

- This state cannot be written as a tensor product of two 1-qubit states
- There is no linear operator that can undo the operation.
- The two qubits will always share the same truth value.

if we measure one of the states and obtain an outcome, it will also reveal the outcome of the other state, i.e. the outcome of one state is the same as the other

Entangled 2-qubit systems: Bell states

$$\begin{aligned}
 |\psi_{0,0}\rangle &= \frac{|00\rangle + |11\rangle}{\sqrt{2}} \\
 |\psi_{0,1}\rangle &= \frac{|10\rangle + |01\rangle}{\sqrt{2}} \\
 |\psi_{1,0}\rangle &= \frac{|00\rangle - |11\rangle}{\sqrt{2}} \\
 |\psi_{1,1}\rangle &= \frac{|01\rangle - |10\rangle}{\sqrt{2}},
 \end{aligned}$$



```

from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi

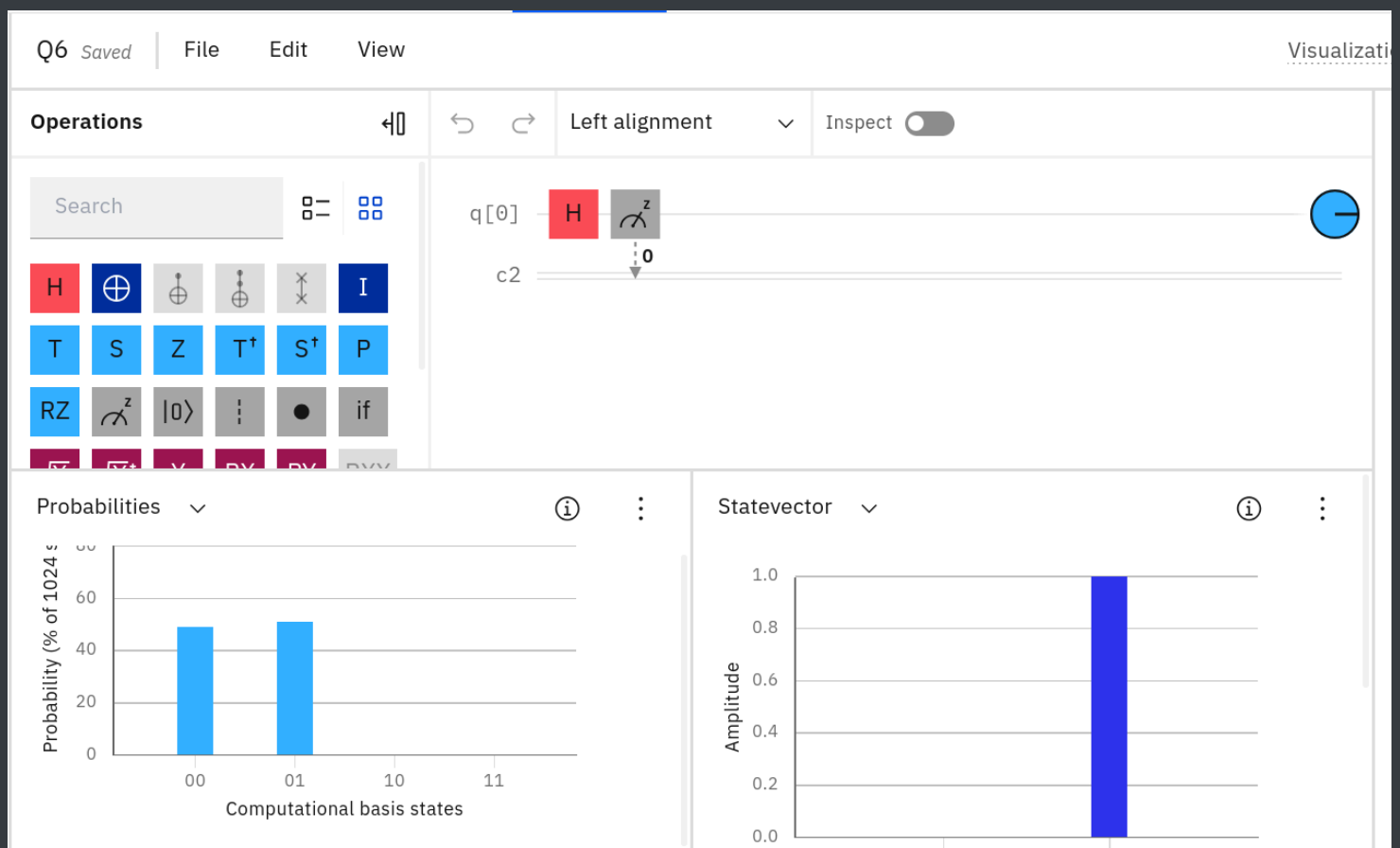
qreg_q = QuantumRegister(2, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)

circuit.h(qreg_q[1])
circuit.cx(qreg_q[1], qreg_q[0])
circuit.x(qreg_q[0])
circuit.z(qreg_q[1])

```

building quantum algorithms

Q6.



--Why does this lead to better 'randomness' than a normal/classical coin toss?

In QM, there are one of two consequences of making a measurement. The accepted consequence until recently has been that the wavefunction collapses into one of the outcomes that is possible according to the probability density. The collapse is the outright destruction of the quantum state to produce this outcome. This leads to a famous paradox called the Schrödinger's cat.

Because it considers all states of outcome, so 0 and 1. Instead of just landing on a random 0 (tails) or 1 (heads), the qubit enters a state called **superposition**. eg . ***Schrödinger's cat paradox***

```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi

qreg_q = QuantumRegister(1, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)

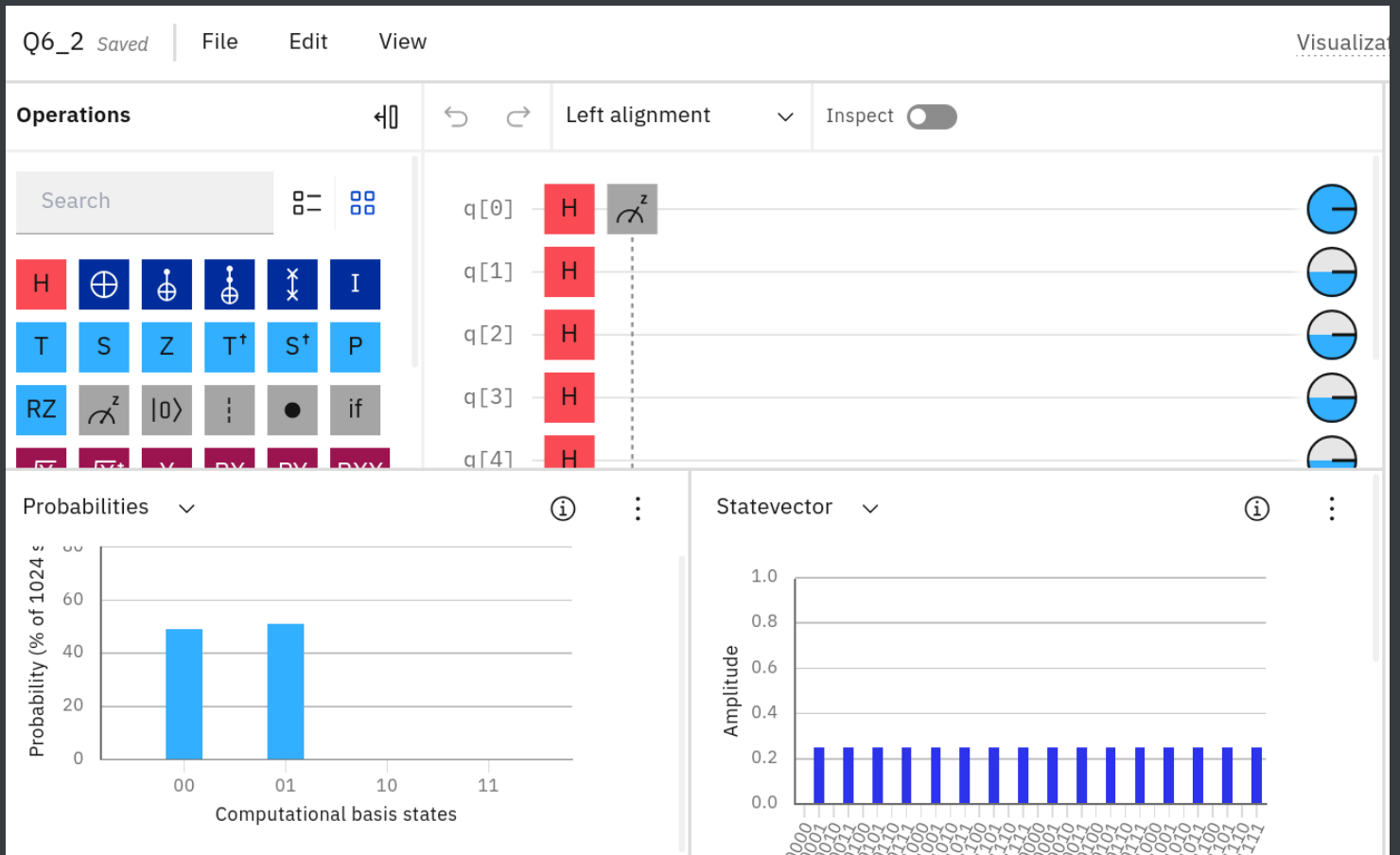
circuit.h(qreg_q[0])
circuit.measure(qreg_q[0], creg_c[0])
```

--How could you extend this to be a random number generator?

As observed, 4 outcomes are shown on the image, eg 00 = 0, 01 = 1, 10 = 2, 11 = 3, so number generator.

and why this would be extended as a number generator?

A qubit would result in '0' or '1', having that idea you can actually put a large number of outputs of qubits together; this would become a string of binaries which could be a number generator.



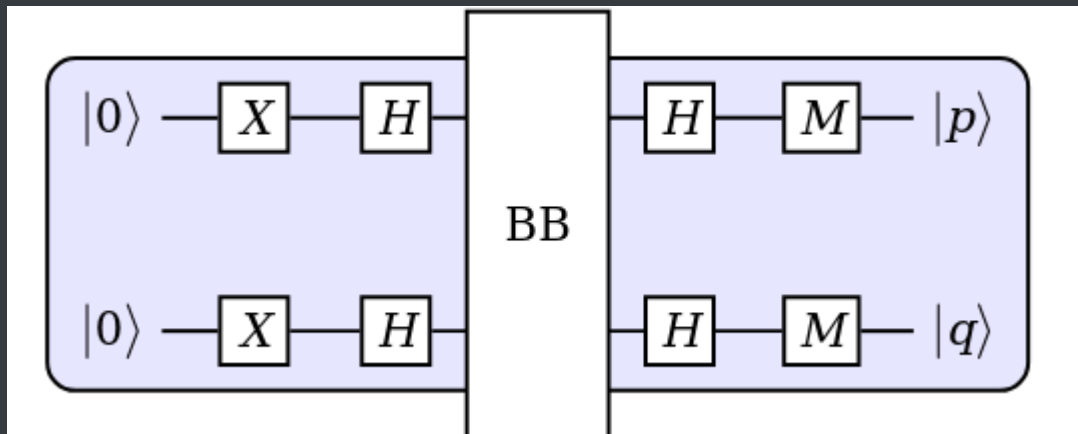
```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi
```

```
qreg_q = QuantumRegister(5, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)
```

```
circuit.h(qreg_q[0])
circuit.h(qreg_q[1])
circuit.h(qreg_q[2])
circuit.h(qreg_q[3])
circuit.h(qreg_q[4])
circuit.measure(qreg_q[0], creg_c[0])
```

Q7.

Implement a solution to the Deutsch oracle problem

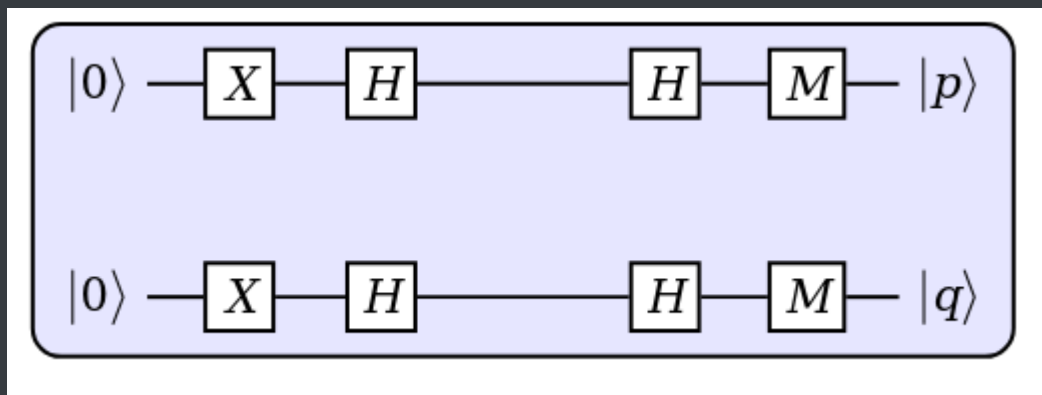


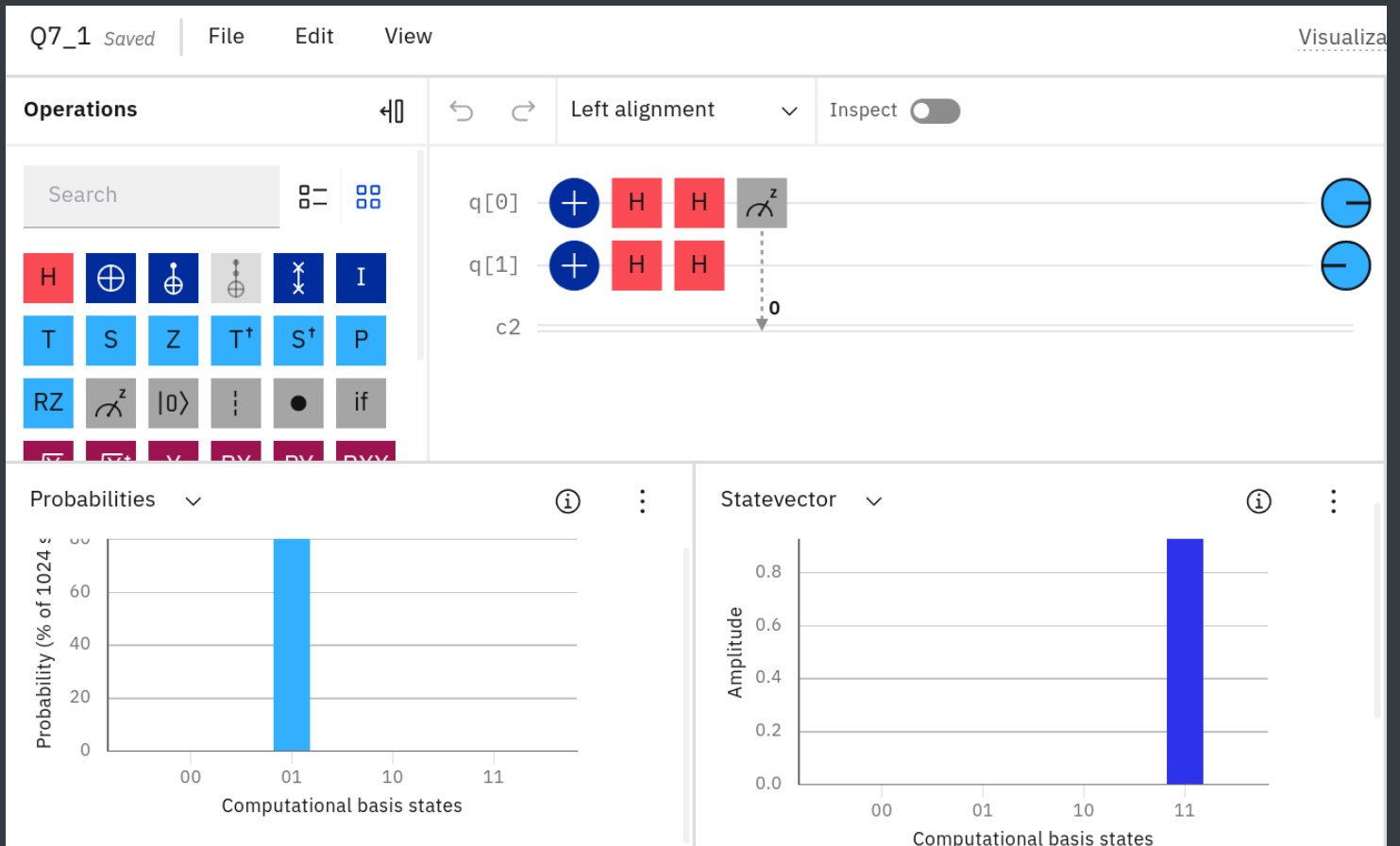
what is Deutsch problem? black box , operations unknown

variable and constant operators.

The variable type of operator should interact with the output and input bits to produce a different tensor product state

Set $|0\rangle$ (left)





```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi
```

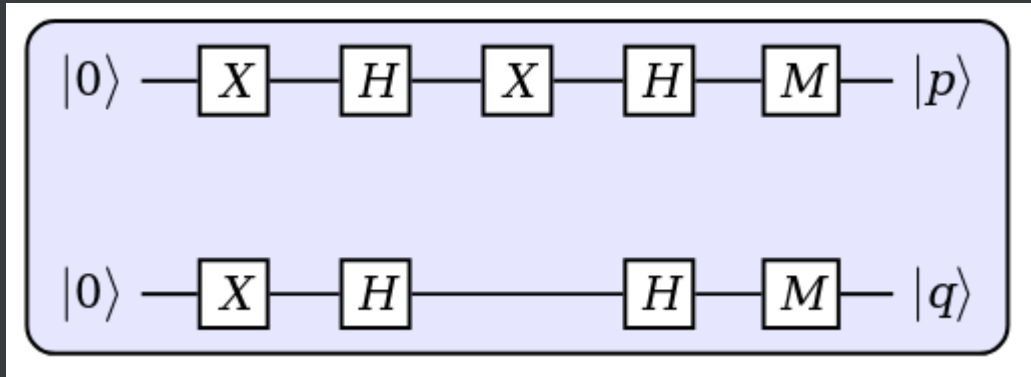
```
qreg_q = QuantumRegister(2, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)
```

```
circuit.x(qreg_q[0])
circuit.x(qreg_q[1])
circuit.h(qreg_q[0])
circuit.h(qreg_q[1])
circuit.h(qreg_q[1])
circuit.h(qreg_q[0])
circuit.measure(qreg_q[0], creg_c[0])
```

Essentially, the Set $|0\rangle$ and Set $|1\rangle$ operations pass through without any changes as our previous circuit constructions have shown

probability density of the result becomes $0^2 + (-1)^2$

Set |1⟩ (right)



```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi
```

```
qreg_q = QuantumRegister(2, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)
```

```
circuit.x(qreg_q[0])
circuit.x(qreg_q[1])
```

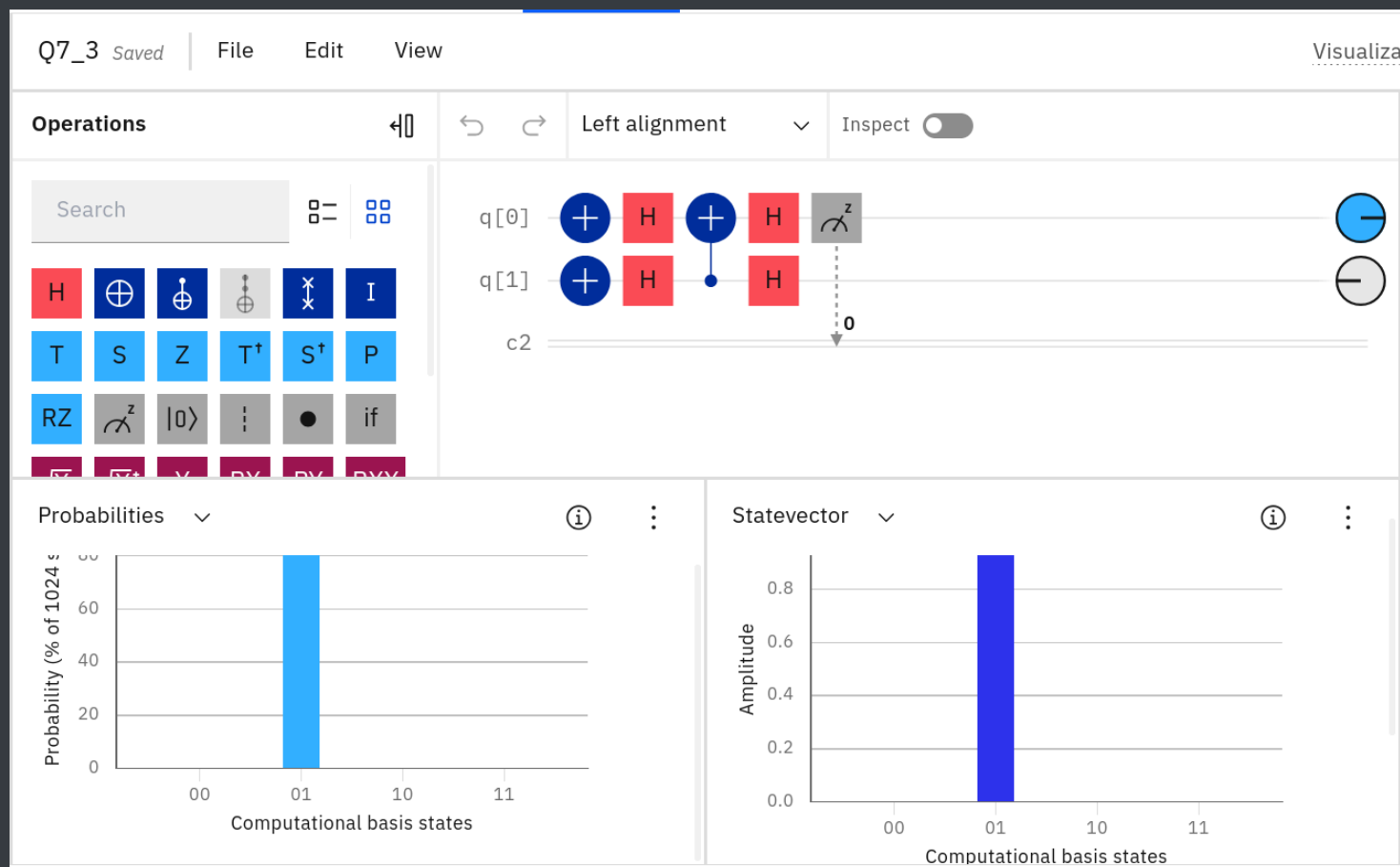
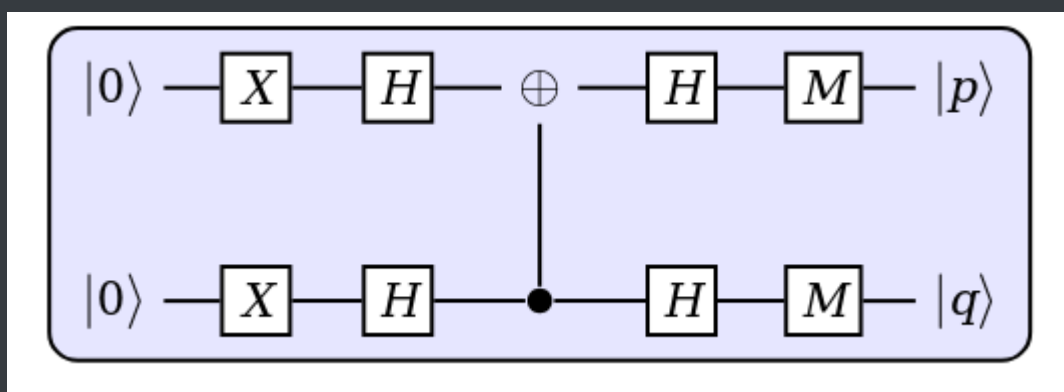
```

circuit.h(qreg_q[1])
circuit.h(qreg_q[0])
circuit.x(qreg_q[0])
circuit.h(qreg_q[1])
circuit.h(qreg_q[0])
circuit.measure(qreg_q[0], creg_c[0])

```

Hence result is always $|11\rangle$.

Identity



```

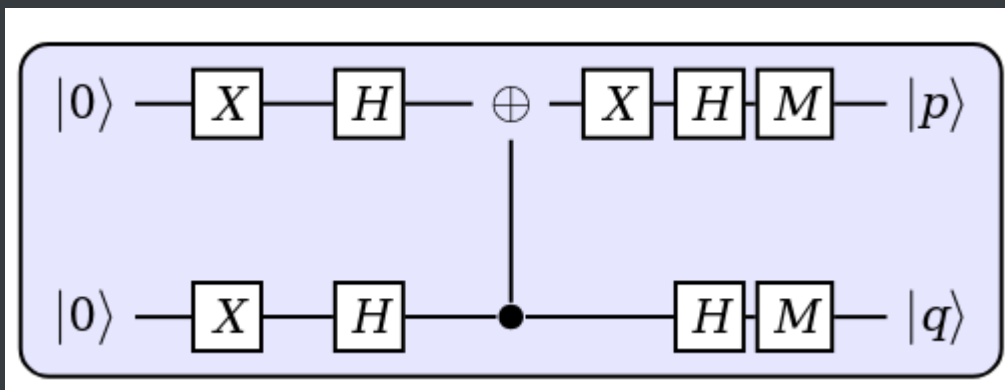
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi

qreg_q = QuantumRegister(2, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)

circuit.x(qreg_q[0])
circuit.x(qreg_q[1])
circuit.h(qreg_q[0])
circuit.h(qreg_q[1])
circuit.cx(qreg_q[1], qreg_q[0])
circuit.h(qreg_q[1])
circuit.h(qreg_q[0])
circuit.measure(qreg_q[0], creg_c[0])

```

Negation





```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from numpy import pi
```

```
qreg_q = QuantumRegister(2, 'q')
creg_c = ClassicalRegister(2, 'c')
circuit = QuantumCircuit(qreg_q, creg_c)
```

```
circuit.x(qreg_q[0])
circuit.x(qreg_q[1])
circuit.h(qreg_q[1])
circuit.h(qreg_q[0])
circuit.cx(qreg_q[1], qreg_q[0])
circuit.h(qreg_q[1])
circuit.x(qreg_q[0])
circuit.h(qreg_q[0])
circuit.measure(qreg_q[0], creg_c[0])
```

Recap Content:

What is QM? wave function

Dirac's notation on classical bits

for 1 bit : 4 operators , reversible = identity / negation

for 2 bits onwards, possible states $|00\rangle$, $|10\rangle$, $|01\rangle$, $|11\rangle$

Tensor product : $|00\rangle = |0\rangle \langle 0|$, also means $|ab\rangle = |a\rangle \langle b| = a \otimes b$

CNOT gate is a reversible XOR gate

A qubit is a bit that is in a superposition of two states

A qubit still collapses into a binary bit, but

like all quantum systems, the final state has a probabilistic outcome given by its probability density.

it has probability density of $|\mu|^2 + |\nu|^2$

Operators of Quantum Circuits: Hadamard Operator

$$H \otimes H = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix} .$$

Hadamard operator is unitary and self-adjoint, we can convert a qubit superposition, like the ones presented in equations (6.48) and (6.49), back into a classical state simply by applying the operator H again.

Quantum circuit: qubit with a 'wire' and the operators as boxes or symbols on these wires.

to flip 0 to 1 use NOT Operator X

identity operation, we require the use of the CNOT

variable operator:

the identity operation as

$$\begin{aligned}
 CHX|00\rangle &= C\left(\left(\begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} \otimes \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}\right)\right) = C\begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ -\frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \\
 &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ -\frac{1}{2} \\ \frac{1}{2} \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ \frac{1}{2} \\ -\frac{1}{2} \end{pmatrix} \\
 &= \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix} = \left(\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \otimes \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}\right) \\
 &= H|01\rangle
 \end{aligned}$$